SC4064 Course Project:

Analysis of Sorting Algorithms using GPU

Group 6

Jodius	U2221134H
Zheng Wen	U2322264J
Zivan	U2421341H

Agenda

1. Introduction
2. Quick Sort
3. Bitonic Sort
4. Merge Sort
5. Radix Sort
6. Conclusion

Introduction

The Problem: Direct GPU conversion of standard sequential / parallel sorting algorithms causes heavy branching memory access, creating massive overhead and destroying parallel advantages. We will be covering QuickSort, Bitonic Sort, Merge Sort and Radix Sort.

The Objective: Systematically evaluate and compare **sequential baselines**, **custom parallel implementations**, and **industry-standard NVIDIA library (Thrust)**.

The Methodology: Apply progressive, hardware optimizations—such as Shared Memory Tiling and Memory Coalescing—to bypass global memory bottlenecks.

Evaluation Metrics: Measure true hardware utilization using **Execution Time**, **Throughput**, and **Roofline Modeling**.

Quick Sort

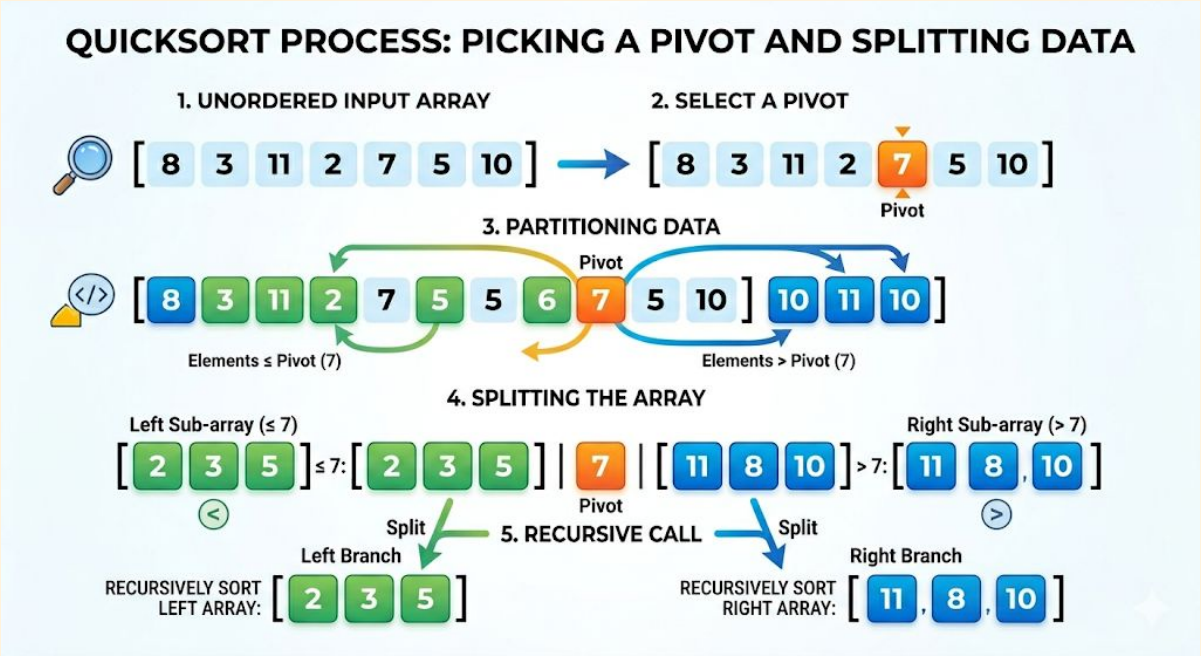
It’s pretty fast - supposedly

Quick Sort Overview

Method: "Divide & Conquer" (Pivot & Partition)

CPU Advantage: Fast (O(N log N)) and cache-friendly, worst case (O(N^2)).

GPU Weakness: Branch-heavy logic causes severe thread divergence.



Quick Sort Results

No. of elements = 2^{18}

Method	Execution time (ms)	Throughput
Baseline (CPU) (2^{18})	19.380	1.35e+07
Std::sort	16.665	1.44e+07
Optimised Parallel GPU	101.852	2.55e+06
Optimised Hardware GPU	1036.380	1.14e+05
Thrust	0.338	2.50e+09

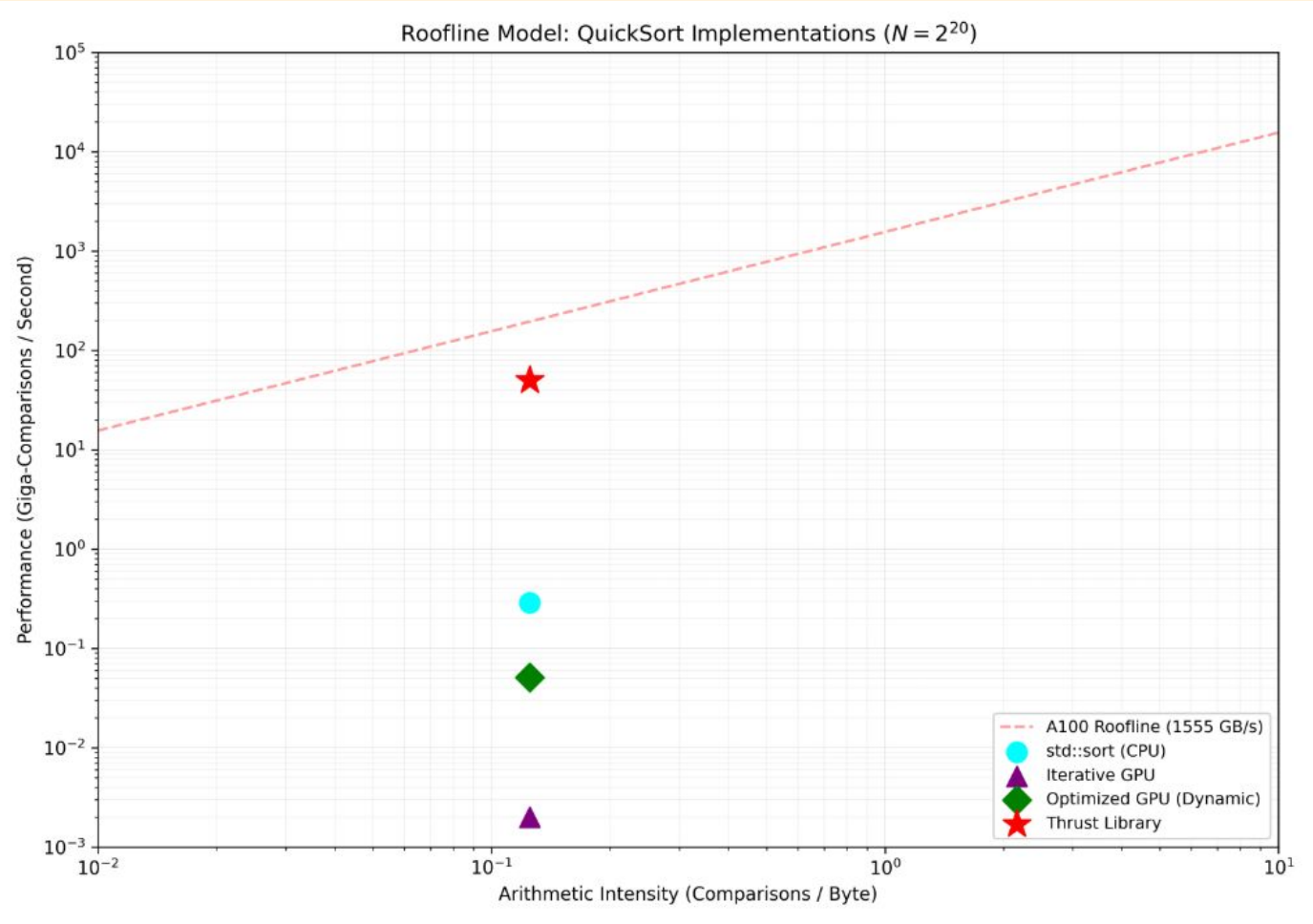
Analysis of results

The CPU Advantage: Sequential, branch-heavy algorithms like QuickSort are inherently suited for CPU cache hierarchies (`std::sort` leads at 16.665 ms).

The Thread Divergence Penalty: The "Optimised Hardware GPU" (1036.380 ms) exposes the flaw of branching on a GPU; divergent warp paths destroy parallel efficiency.

Task Management over Memory Management: The "Optimised Parallel GPU" (101.852 ms) improved performance 10x by offloading recursion dynamically, but remains capped by kernel launch overhead.

The Algorithmic Reality: Thrust’s massive leap (0.338 ms) confirms that peak GPU saturation requires abandoning QuickSort for architecture-native algorithms. (Eg Radix Sort)


Roofline Model: QuickSort Implementations ($N = 2^{20}$)
Performance (Giga-Comparisons / Second)
Arithmetic Intensity (Comparisons / Byte)
10^{5}
10^{4}
10^{3}
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-2}
10^{-1}
10^{0}
10^{1}
A100 Roofline (1555 GB/s)
std::sort (CPU)
Iterative GPU
Optimized GPU (Dynamic)
Thrust Library

Bitonic Sort

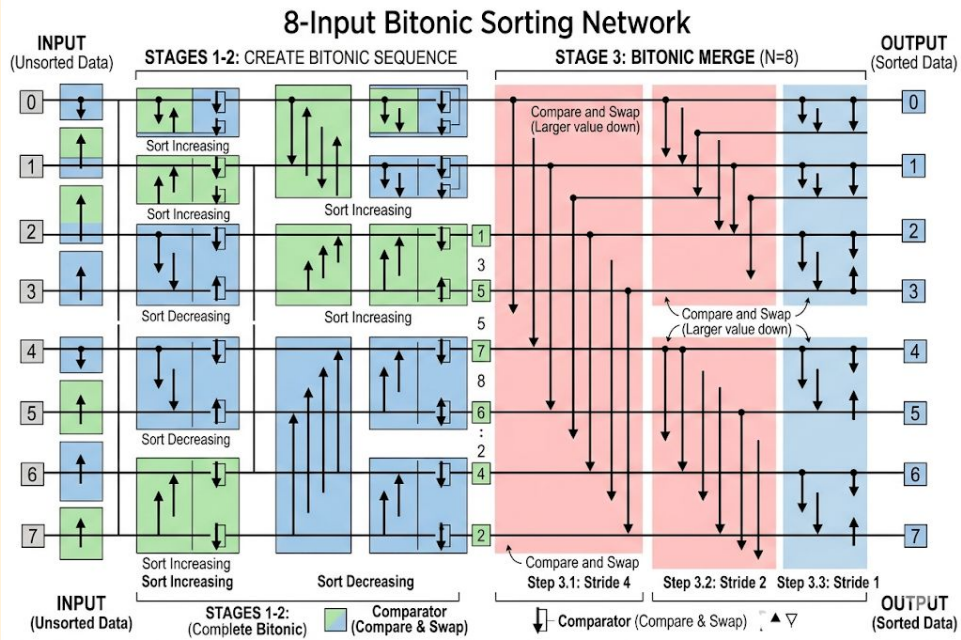
Recursive parkour: jump up, jump down, and somehow land in order.

Bitonic Sort Overview

Method: A parallel sorting algorithm that builds and merges "bitonic" (up-and-down) sequences. Operates on a rigid, mathematically predefined network of comparisons.

CPU Weakness: High math overhead (N log^2 N).

GPU Advantage: 100% branchless and predictable.



Bitonic Sort Results

No. of elements = 2^{20}

Method	Execution time (ms)	Throughput
Baseline (CPU)	1,379,870	7.59e+02
Naive GPU	1.578	6.64e+08
Tiled GPU	1.311	8.00e+08
Warp Shuffle	1.402	7.48e+08
Thrust	0.420	2.46e+09

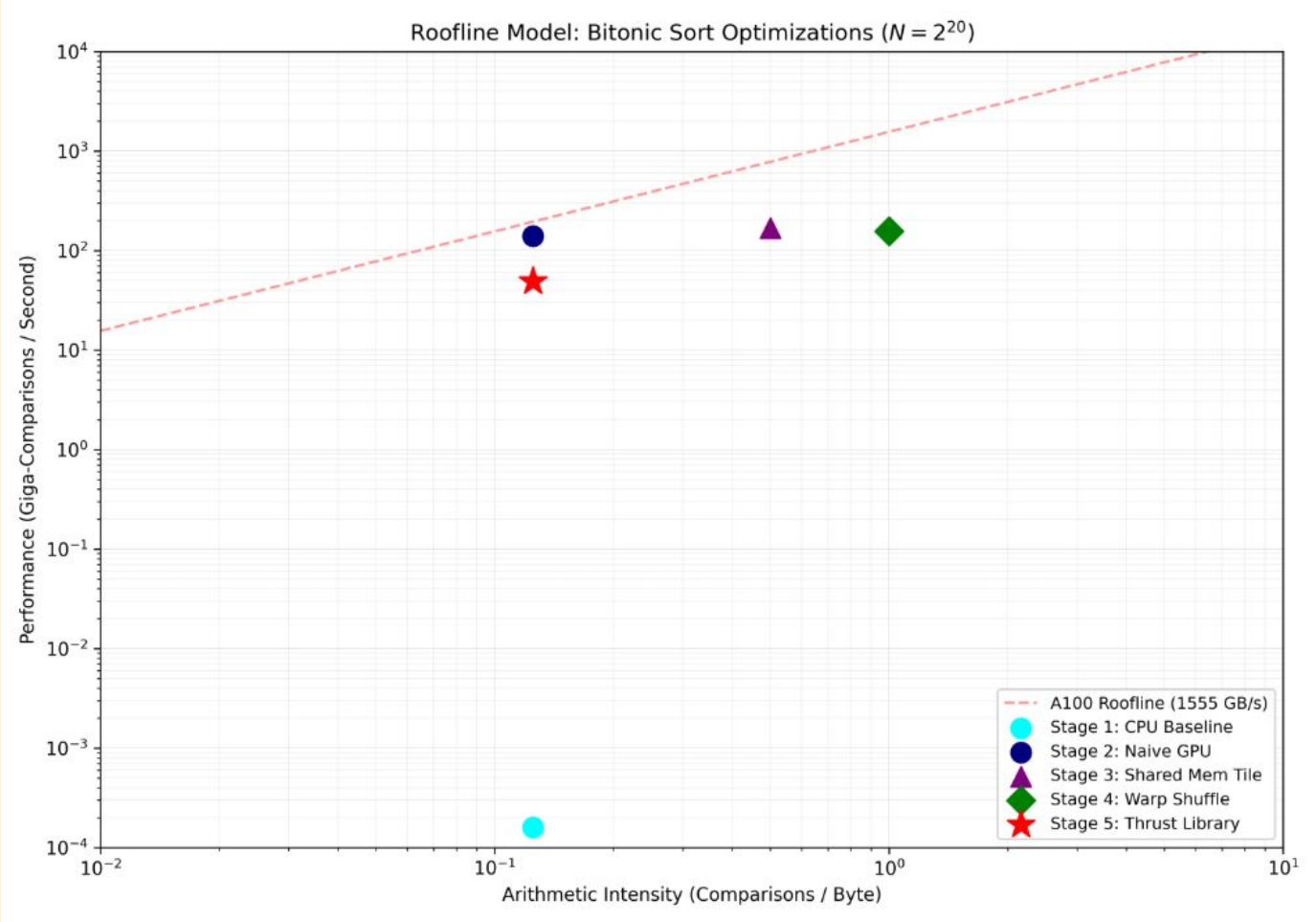
Analysis of results

The CPU's Mathematical Ceiling: The O(N log^2 N) complexity crippled the sequential CPU baseline, taking nearly 23 minutes (1,379,870 ms).

The Power of Data-Parallelism: The "Naive GPU" (1.578 ms) instantly unlocked massive performance because branchless sorting networks guarantee perfect lockstep execution.

Hitting the Memory Wall: Deep hardware optimizations—"Tiled GPU" (1.311 ms) and "Warp Shuffle" (1.402 ms)—successfully bypassed VRAM latency but saturated the physical hardware bandwidth.

The "Thrust" Revelation: Thrust’s completion time (0.420 ms) would require physically impossible bandwidth for a Bitonic network, proving it uses an entirely different algorithm (Radix Sort).


Roofline Model: Bitonic Sort Optimizations ($N = 2^{20}$)
Performance (Giga-Comparisons / Second)
Arithmetic Intensity (Comparisons / Byte)
10^{4}
10^{3}
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-2}
10^{-1}
10^{0}
10^{1}
A100 Roofline (1555 GB/s)
Stage 1: CPU Baseline
Stage 2: Naive GPU
Stage 3: Shared Mem Tile
Stage 4: Warp Shuffle
Stage 5: Thrust Library

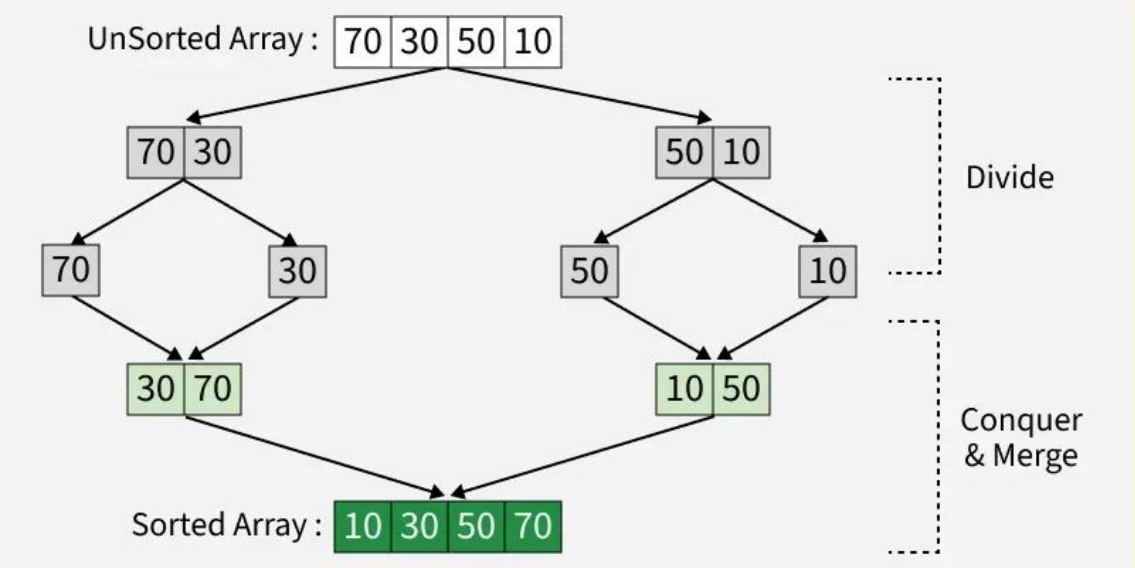
Merge Sort

The ultimate 'break it until you fix it' approach.

Merge Sort Overview

Divide & Conquer:
Divide recursively until size 1
Each subarray is sorted individually and merged back

O(NlogN) Always
Regardless of input distribution



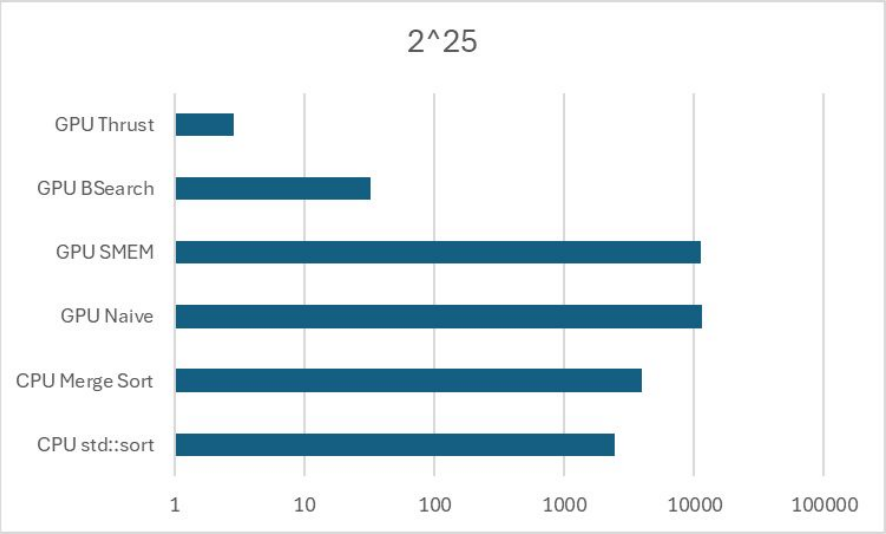
Merge Sort Results

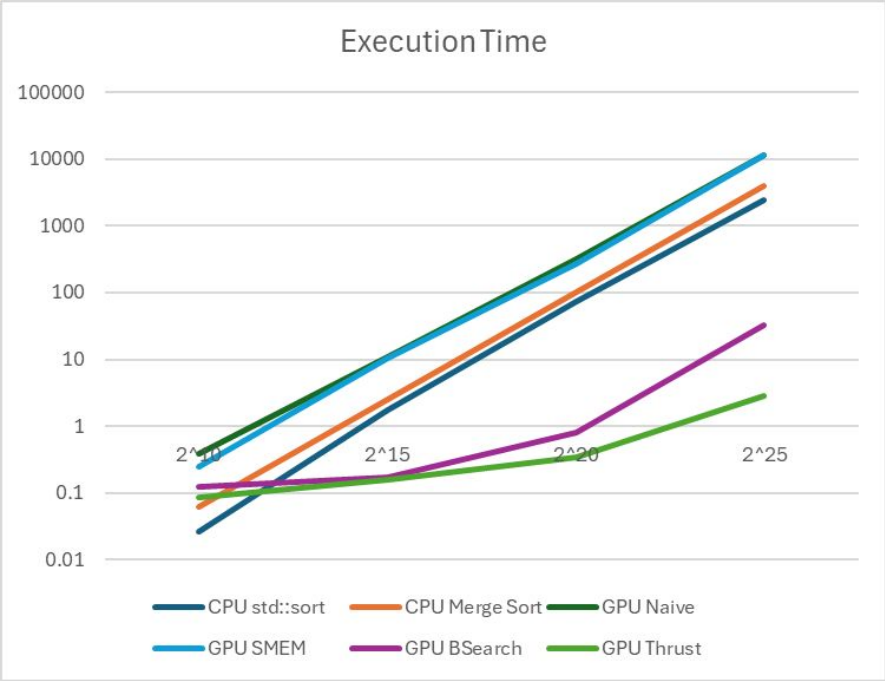
Execution Time (ms):

Algorithm	2^10	2^15	2^20	2^25
CPU std::sort	0.026	1.710	73.106	2470.760
CPU Merge Sort	0.061	2.467	103.106	3962.277
GPU Naive	0.389	10.814	319.138	11436.662
GPU SMEM	0.246	10.478	266.161	11376.401
GPU BSearch	0.124	0.171	0.812	32.465
GPU Thrust	0.085	0.158	0.335	2.868

Throughput:

Algorithm	2^10	2^15	2^20	2^25
CPU std::sort	40.1	19.2	14.3	13.6
CPU Merge Sort	16.7	13.3	10.2	8.5
GPU Naive	2.6	3.0	3.3	2.9
GPU SMEM	4.2	3.1	3.9	3.0
GPU BSearch	8.3	191.2	1291.4	1033.5
GPU Thrust	12.1	207.0	3132.4	11698.8


2^25
GPU Thrust
GPU BSearch
GPU SMEM
GPU Naive
CPU Merge Sort
CPU std::sort
1
10
100
1000
10000
100000


Execution Time
100000
10000
1000
100
10
1
0.1
0.01
2^10
2^15
2^20
2^25
CPU std::sort
CPU Merge Sort
GPU Naive
GPU SMEM
GPU BSearch
GPU Thrust

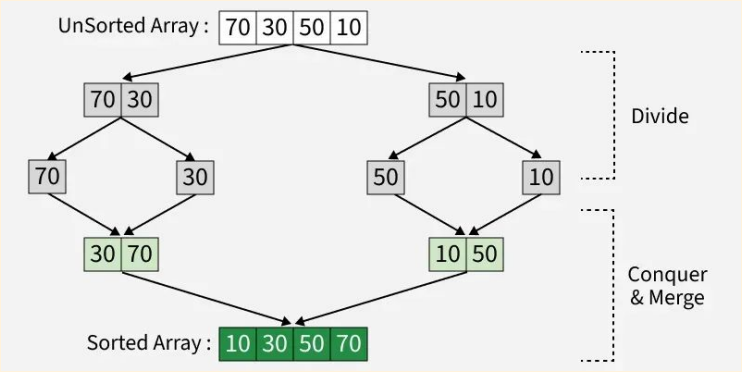
Naive GPU Merge

At 2^{20}

11436.662ms (GPU Naive) VS 3962ms (Standard CPU merge sort)

Why did performance degrade severely?

- Early passes allowed for massive parallelism
- Last pass was done by a single thread
  - Last 5 passes accounted for >99% of total execution time



Pass	Width	Merges	Time (ms)	Elem/s	BW (GB/s)
1	1	16,777,216	0.288	9.32e+10	932.4
2	2	8,388,608	0.444	7.55e+10	604.0
3	4	4,194,304	0.740	4.53e+10	362.6
4	8	2,097,152	1.047	3.21e+10	256.5
5	16	1,048,576	3.720	9.02e+09	72.2
6	32	524,288	6.517	5.15e+09	41.2
8	128	131,072	7.312	4.59e+09	36.7
9	256	65,536	2.549	1.32e+10	105.3
10	512	32,768	2.470	1.36e+10	108.7
20	524,288	32	571.222	5.87e+07	0.470
23	4,194,304	8	1652.4	2.03e+07	0.162
24	8,388,608	2	2469.8	1.36e+07	0.109
25	16,777,216	1	3987.2	8.42e+06	0.067

Shared Memory Optimization

Implementation

- Sort 512-element tiles entirely in shared memory
  - Shared memory per block constraint
  - Threads per block constraint (1024)
- Saves first 9 global memory passes

Analysis

- Improvements negligible as the first few passes are the fastest
  - 11,376 ms vs 11,437ms
- The final passes dominate runtime
  - Shared memory unable to be used as it is too small

GPU Binary Search

Problem with each thread doing a sequential merge:

- Each thread has to sort larger and larger lists
- Every output element depends on previous one

BSearch Overview:

- Each thread is in charge of 1 output value
- Instead of “What comes next?”, each thread asks “What belongs in my slot?”

GPU Binary Search

- Each thread in charge of filling 1 output position
- E.g. Thread 5 fills output[5]

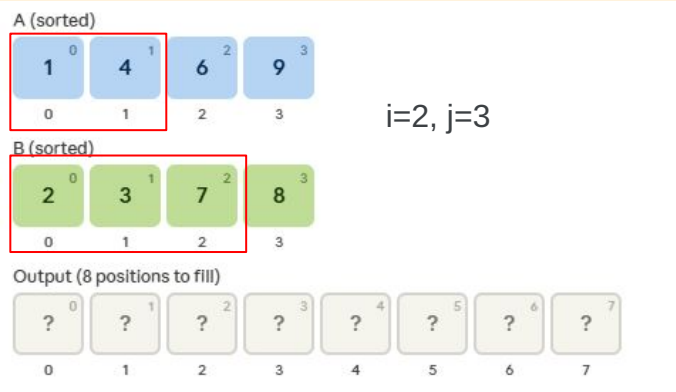
How?

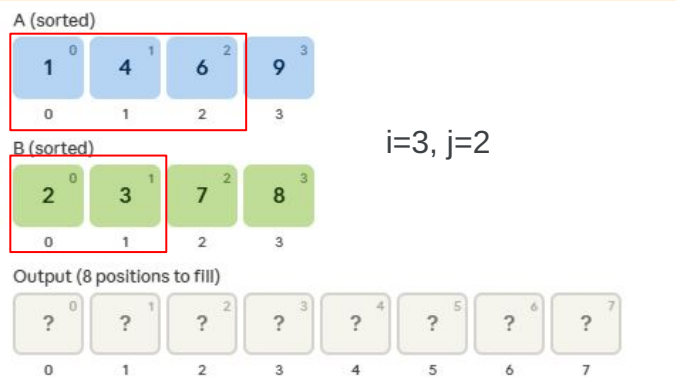
- Let i be number of elements from A in output[0-4]
- Let j be number of elements from B in output[0-4] = *5-i*
- If we know i & j, element 5 is simply *min(A[i], B[j])*

Using <u>Binary Search</u>:

1) Set i to be the midpoint = 2
2) Test condition: A[i] < B[j-1]?
   a) Is the next element from A smaller than the last element used from B?
3) If yes, A[i] should have come earlier, increase i
4) If no, split is valid (or overshot), decrease i

Result: i*=3, j*=2 → A[3]=9 VS B[2]=7 → output[5]=7



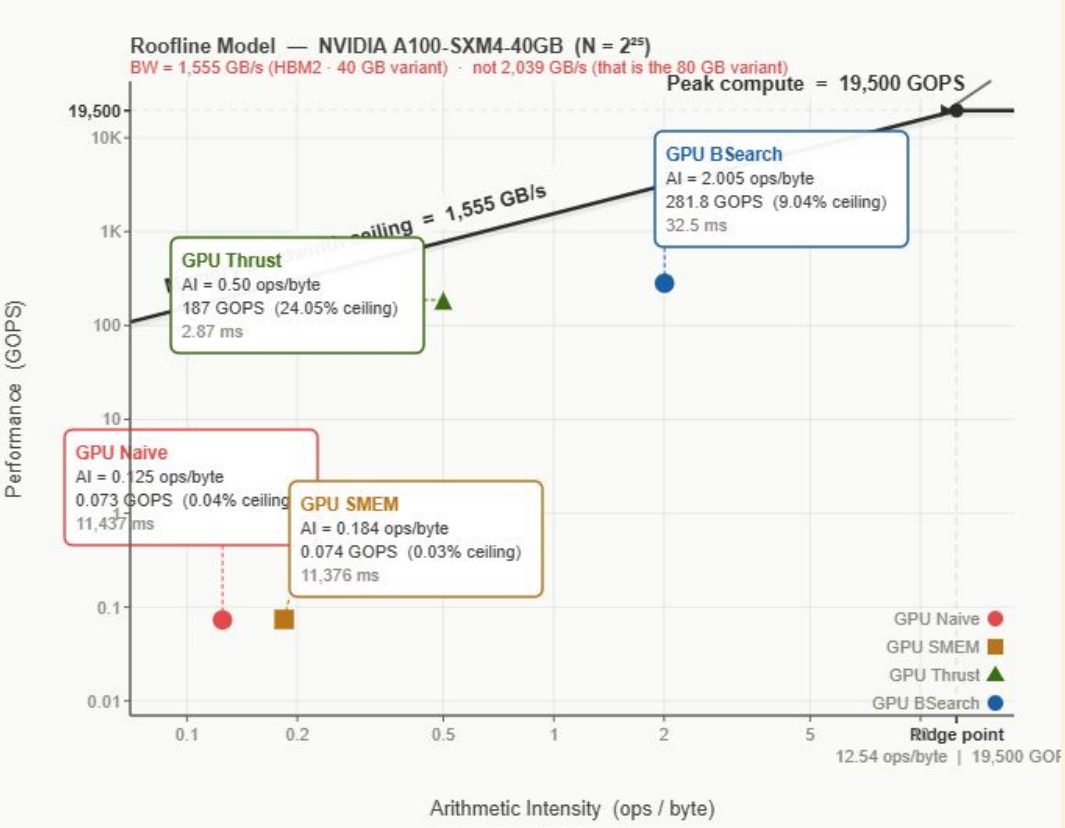


GPU Binary Search Analysis

- Instead of 1 thread scanning 2*width elements to produce merged output, all N threads execute in parallel
- Binary search O(log width) to find single output independently of other threads
- Eliminates the previous problem where at higher widths, each thread is doing more and more work, and work becomes more sequential
- Closest to Thrust library sort
  - For 2^{25}: 32.4ms VS 2.8ms
  - 100+ Times speed up over CPU Merge Sort

Width	GPU Naive: threads x work/thread	GPU BSearch: threads x work/thread
width = 512	65,536 threads x 1,024 sequential ops	33.5M threads x 9 ops (log2 512)
width = 4,194,304	8 threads x 8.4M sequential ops	33.5M threads x 22 ops (log2 4M)
width = 16,777,216	1 thread x 33.5M sequential ops	33.5M threads x 24 ops (log2 16M)

Roofline Model Analysis



Radix Sort

The sorting algorithm that doesn't actually know how to compare two numbers.

Phase 1: Naive Implementation

Outer Loop: 1s, 10s, 100s, etc

Inner Loop: 10 Digit buckets (0 - 9)

Step 1: Masking
0 if digit != bucket
1 if digit = bucket

In global memory
Done 10x

Step 2: Parallel Prefix Sum

Checks the entire size of the input

Step 3: Scatter Kernel

Phase 2: Shared Memory Tiling

Outer Loop: 1s, 10s, 100s, etc

<u>Now does every digit in one go</u>

<u>New</u> Step 1: Block Histogram Generation

In shared memory
Done 1x

<u>Same</u> Step 2: Parallel Prefix Sum (<u>Much faster</u>)

Sums block-specific offset

<u>New</u> Step 3: Local Reorder and Scatter

Uses 2x shared memory variables:

- Block offset
- Tracking of thread digits

Each thread writing all over the place

Phase 3: Memory Coalescing

Outer Loop: 1s, 10s, 100s, etc

<u>Now does every digit in one go</u>

<u>Same</u> Step 1: Block Histogram Generation

In shared memory
Done 1x

<u>Same</u> Step 2: Parallel Prefix Sum (<u>Much faster</u>)

Sums block-specific offset

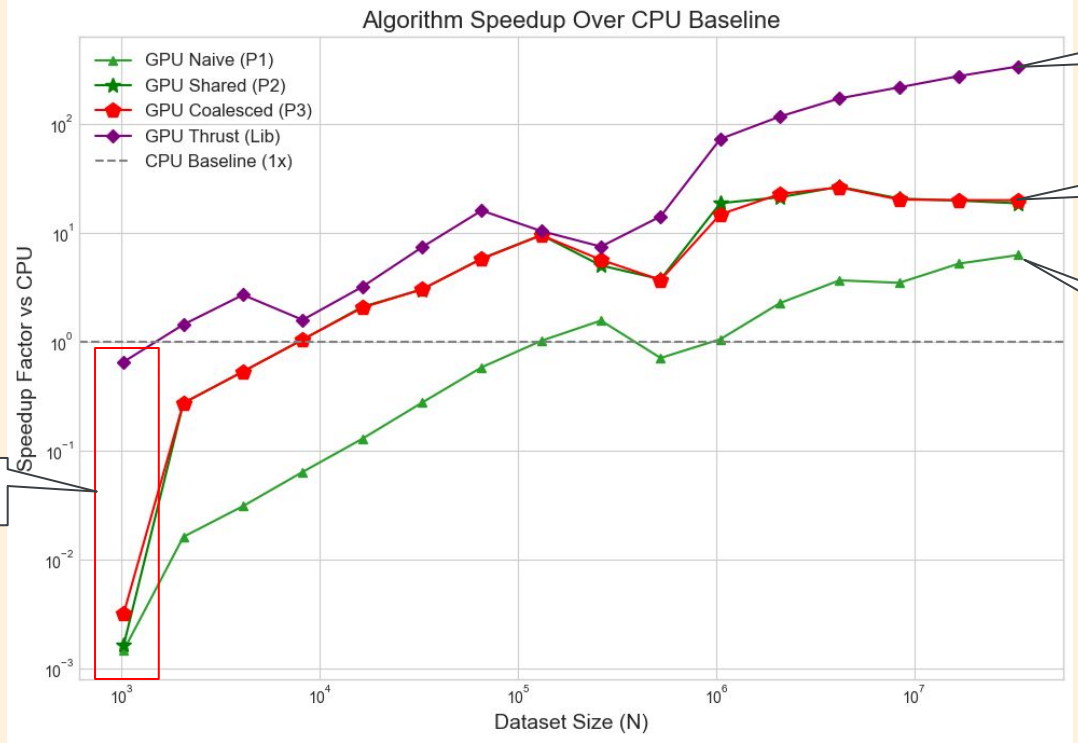
New Step 3: Coalesced Reorder and Scatter

Uses 2x shared memory variables:

- Block offset
- Tracking of thread digits

Sorts starting threads from 0, then 1, etc

Results: Performance Scaling



Thrust:
~337x Increase

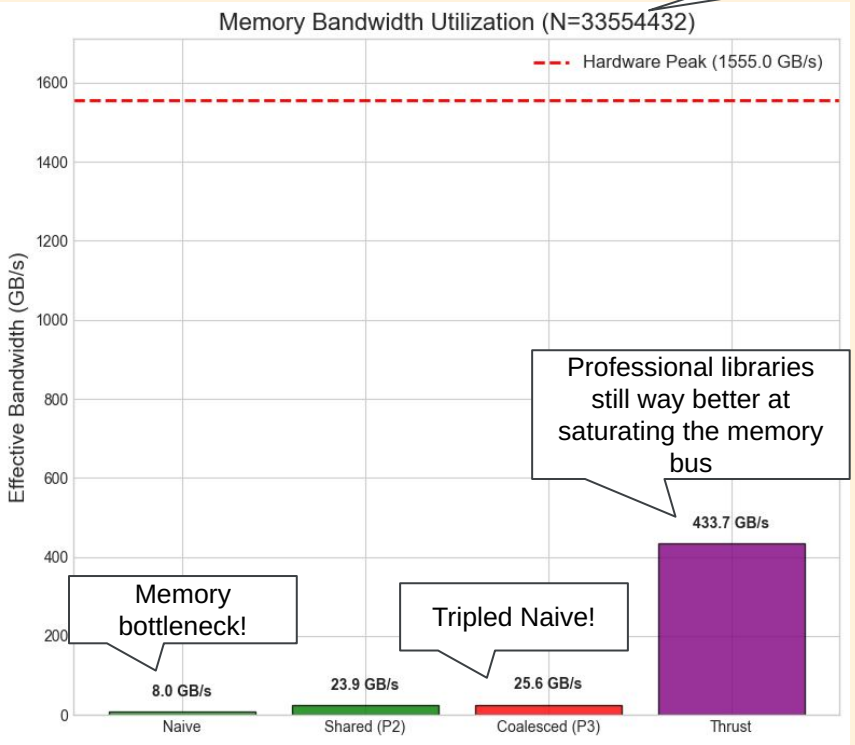
Phase 2 + 3:
~20x Increase

Naive:
~6x Increase

CPU better at small sizes

Results: Memory Bandwidth & The Roofline

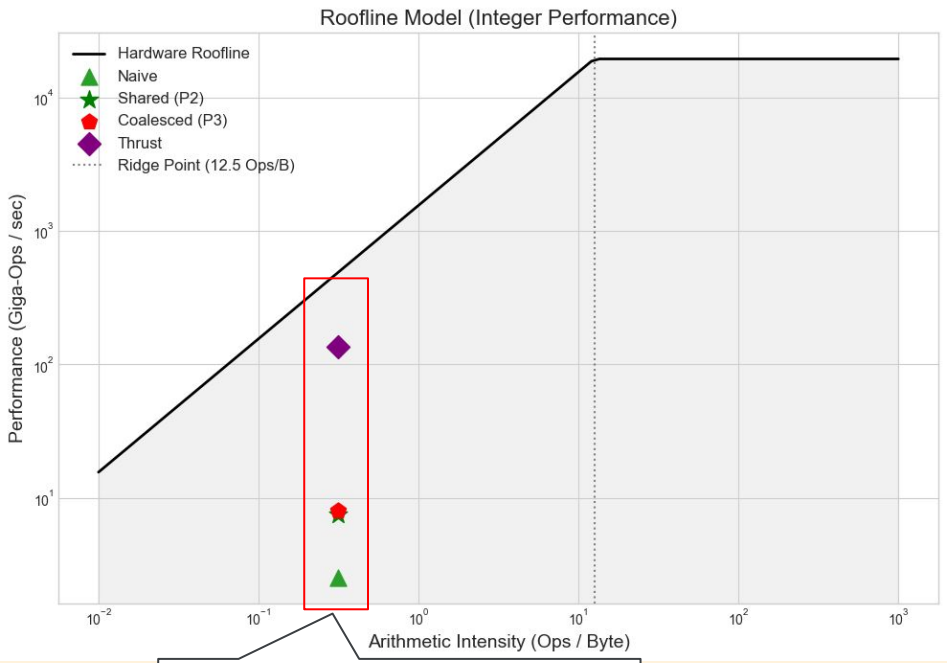
Largest Dataset:
2^{25} Data



Memory bottleneck!

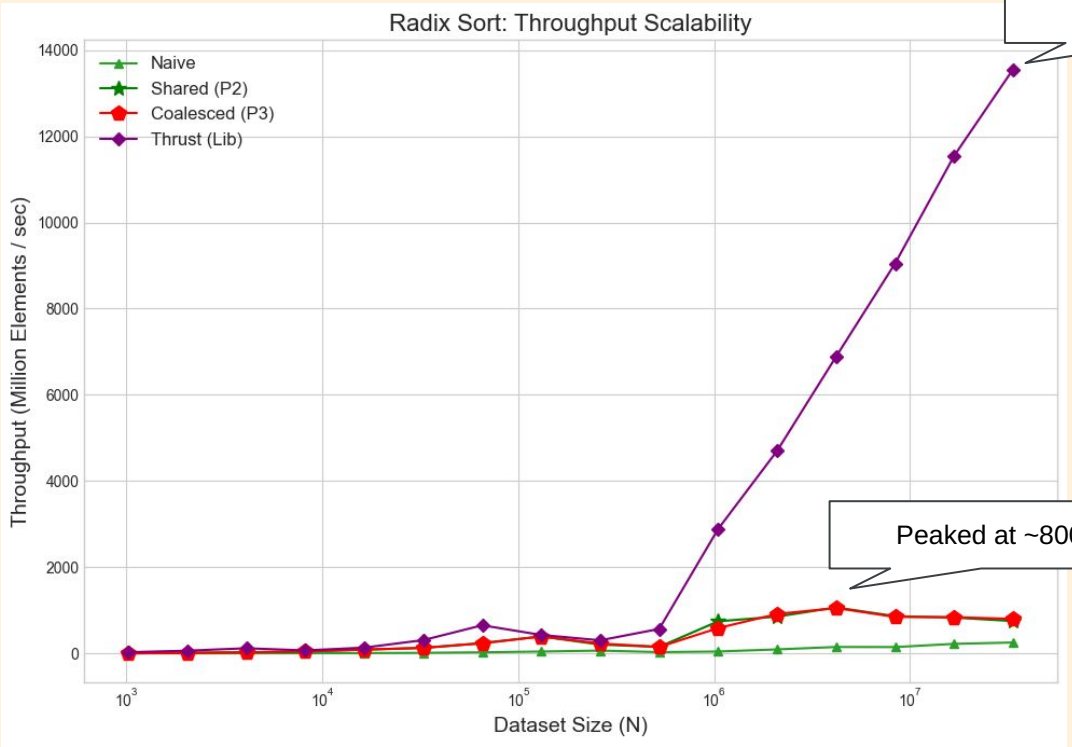
Tripled Naive!

Professional libraries still way better at saturating the memory bus



Memory Bound:
Moving memory is constraint,
not compute speed

Results: Scalability & Industry Benchmarks



Still going strong

Potential additional tuning they did:

- Warp-Level Shuffles
- Register Tiling
- Kernel Fusion

Peaked at ~800M/s

Conclusion

What did we learn? What did you learn?

Key takeaways

Quick Sort

1

The Divergence Trap

Sequential logic cannot be brute-forced onto parallel hardware.

Algorithms with heavy branching logic suffer from **thread divergence**, where the hardware is forced to serialize operations.

Merge Sort
Bitonic Sort

2

The Parallel Re-Architecture

Adopt either **branchless sorting networks** like Bitonic Sort
or implement **advanced parallel merge strategies** like Binary Search Merging to ensure all threads remain productive simultaneously.

Radix Sort

3

The Memory Barrier

The bottleneck is not the math—it is the move. Even a "perfectly parallel" design is ultimately capped by the physical limits of the memory bus.
Reaching peak performance requires a **deep mastery of the memory hierarchy**

To get the most out of a GPU,
you can't just be a coder,
you have to be a **hardware architect**.

Thank you!